

Sharding can be basically done in two ways, using *vertical partitioning* or *horizontal partitioning*.

- In vertical partitioning, we store data in different tables having different documents *in different databases*.
- In horizontal partitioning, we store documents of the same table *in separate databases*. The range shard algorithm is a dynamic algorithm that determines the breaking point of the table and stores data in different shards based on the calculation.

Range-based sharding

In the range sharding algorithm, we use a service called locator to determine the entries in a particular table. The locator service finds out the data using range queries and hence it becomes faster than others. If you do not have a range or some kind of indicator to know which data belongs to which shard in which server, you will need to look over every database to find the particular document, which no doubt turns into a very slow process.

Replication in RethinkDB

Replication provides a copy of data in order to improve performance, availability, and failover handling. Each shard in RethinkDB can *contain a configurable number of replicas*. A RethinkDB instance (node) in the cluster can be used as a *replication node for any shard*. You can always change the replication from the RethinkDB web console.

Indexing in RethinkDB

RethinkDB uses the primary key by default to index a document in a table. If the user does not provide primary key information during the creation of the table, RethinkDB uses its default name ID.

The default-generated primary key contains information about the shard's location in order to directly fetch the information from the appropriate shard. The primary key of each shard is indexed using the B-Tree data structure.

RethinkDB also provides the *secondary key* and *compound key (combination of keys) features*. It even provides **multi-index features** that allow you to have **arrays of values acting as keys**, which again can be single compound keys.

Like SQL DB: Having system-generated keys for primary is very efficient and fast, because the query execution engine can immediately determine on which shard the data is present. Hence, there is no need for extra routing, while having a custom primary key, say an alphabet or a number, may force RethinkDB to perform more searching of data on various clusters. This slows down the performance. You can always use secondary keys of your choice to perform further indexing and searching based on your application needs.

Automatic failover handling in RethinkDB

RethinkDB provides automatic failover handling in a multi-server configuration where multiple replicas of a table are present.

However, there are some requirements that must be met before considering automatic failover handling:

- The cluster must have three or more nodes (RethinkDB servers)
- The table must be set to have three or more replicas set with the voting option
- During failover, the majority of replicas (greater than half of all replicas) for the table must be online

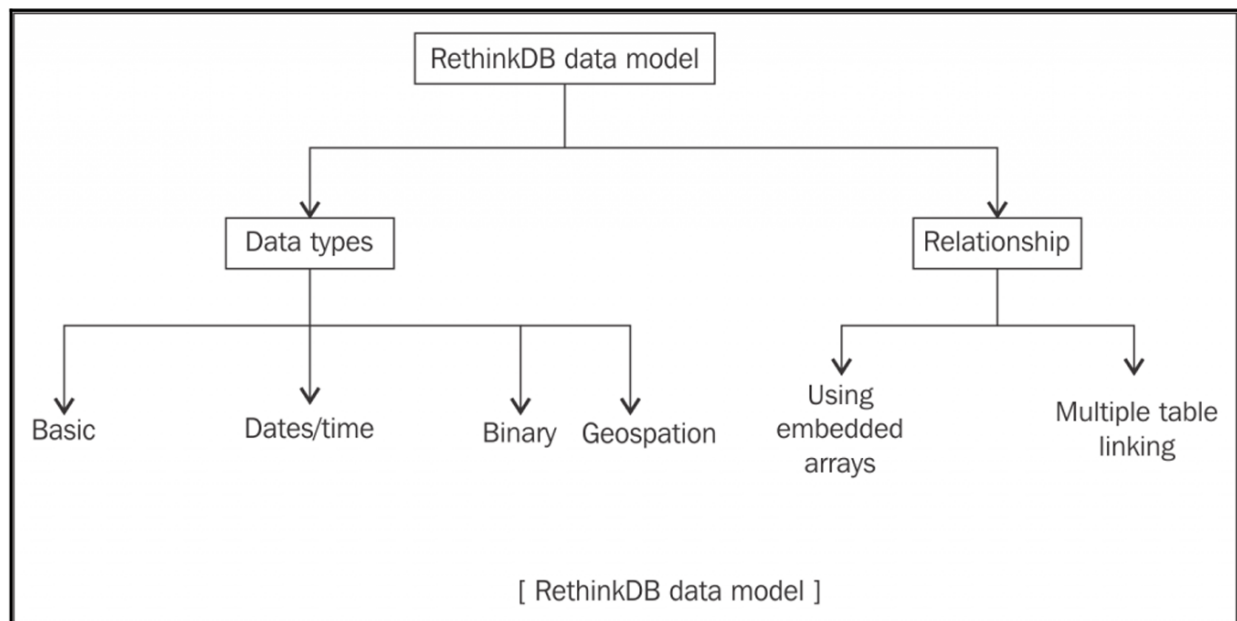
Every table, by default, has a primary replica created by RethinkDB. You can always change that using the **reconfigure() command**. In case of failure of the *primary replica of the table*, as long as more than half of the replicas with voting option are available, one of them will be internally selected as the primary replica. There will be a slight offline scenario while the selection is going on in RethinkDB, but that will be very minor and no data will be lost.

The RethinkDB data model

The RethinkDB data model consists of two main components:

- RethinkDB data types
- RethinkDB model relationship

RethinkDB data types are further classified into basic data types, dates, binary objects, and geospatial queries.



RethinkDB data types

RethinkDB provides six basic data types; they are represented in tabular format, as shown in the following table:

Data type	Description
Numbers	Numbers are stored using double-precision floating-point numbers.
Strings	Strings are stored in UTF-8 standard.
Objects	Stored as a key-value pair with standard JSON format.
Arrays	Stored as a list of elements. It supports 100,000 elements by default.
Binary	Binary objects includes files, images, and other binary data.
Date and time	RethinkDB stores date and time with millisecond precision.
Booleans	True and false values.
Null	Stores null values.

RethinkDB also provides extended data types supported by it. They are as follows:

Tables: These are RethinkDB tables. You can insert and delete documents from them with proper indexing.

Streams: Streams provide the transfer of large amounts of data in chunks so that the system won't reach the buffer overflow limit. They are *loaded in a lazy fashion*. Streams provide the navigation point to a chunk, called cursor, which you can use to traverse the result set. You can club all of them once the streams are collected using the cursor. This makes it easy to read large amounts of data. *Streams are read-only operations.*

Selections: Selections are subsets of data, such as 10 documents out of 1,000 from a table. There are two types of selections, one with objects and one with streams. The difference between the two is that objects can be writable while streams are read-only. You can pass an object selected by, say, the `get()` command to other commands to manipulate the data.

Date and time: Date and time are stored in RethinkDB *with millisecond precision*, along with time zones. Currently, minute-precision time offsets from UTC are supported. RethinkDB internally calculates the time zone difference; hence you don't need to do it at the client end.

Geospatial queries in RethinkDB

RethinkDB provides and supports geospatial features to help you build location-based applications. By using geospatial queries, you can easily parse, convert, and perform lots of operations, such as computing the distance between two locations, finding an intersecting location, and many more. RethinkDB stores all geographical information in the GeoJSON standard format.

RethinkDB uses a geometry object to perform geographical queries. Geometry objects are *derived by plotting two-dimensional objects* such as lines and points *on a sphere in three dimensional space*.

```
r.table('demo').insert([
  {
    name : "Mumbai",
    location : r.point(19.0760, 72.8777)
  }
])
```

Supported data types

Geospatial data types and functions are derived from three geometric object data types: points, lines, and polygons.

By using these three, RethinkDB provides various geospatial functions such as `circle()`, `intersect()`, `getNearest()`, and so on.

RethinkDB model relationships

RethinkDB, besides being a NoSQL database, provides one of the most requested features by SQL developers, that is, JOINS. RethinkDB allows you to model and structure your data in such a way that allows you to perform JOINS over it. There are two ways to model relationships in RethinkDB:

- By using *embedded arrays* in the document
 - By linking *documents stored* in multiple tables
-

Embedded arrays

Embedded arrays basically means using an array of objects in the table in each document. Here is a simple example:

```
{
  "id": "7644aaf2-9928-4231-aa68-4e65e31bf219",
  "name": "Shahid",
  "chapters": [
    {"title": "Chapter 01", "pages": "30+"},
    {"title": "Chapter 02", "pages": "30+"}
  ]
}
```

Here, the "chapters" key contains the array of objects, and each object contains the name of the chapter and the page count. In order to perform the query, here is how we do it in RethinkDB.

To display all chapters from table:

```
r.db("book").table("chapters").run()
```

To get all the chapters written by a particular author; we can do that by passing the author ID:

```
r.db("book").table("chapters").get('7644aaf2-9928-4231-aa68-4e65e31bf219').run()
```

Here, we are passing the ID of the document, which is a system-generated primary key ID by RethinkDB. Upon passing it, we can access a single document from the table. This is a very basic way of modelling and is not recommended for production use.

Merits of embedded arrays

The following merits are as follows:

- *Easy to design and query*
- *Updates on a single document will automatically update chapters and authors both.*

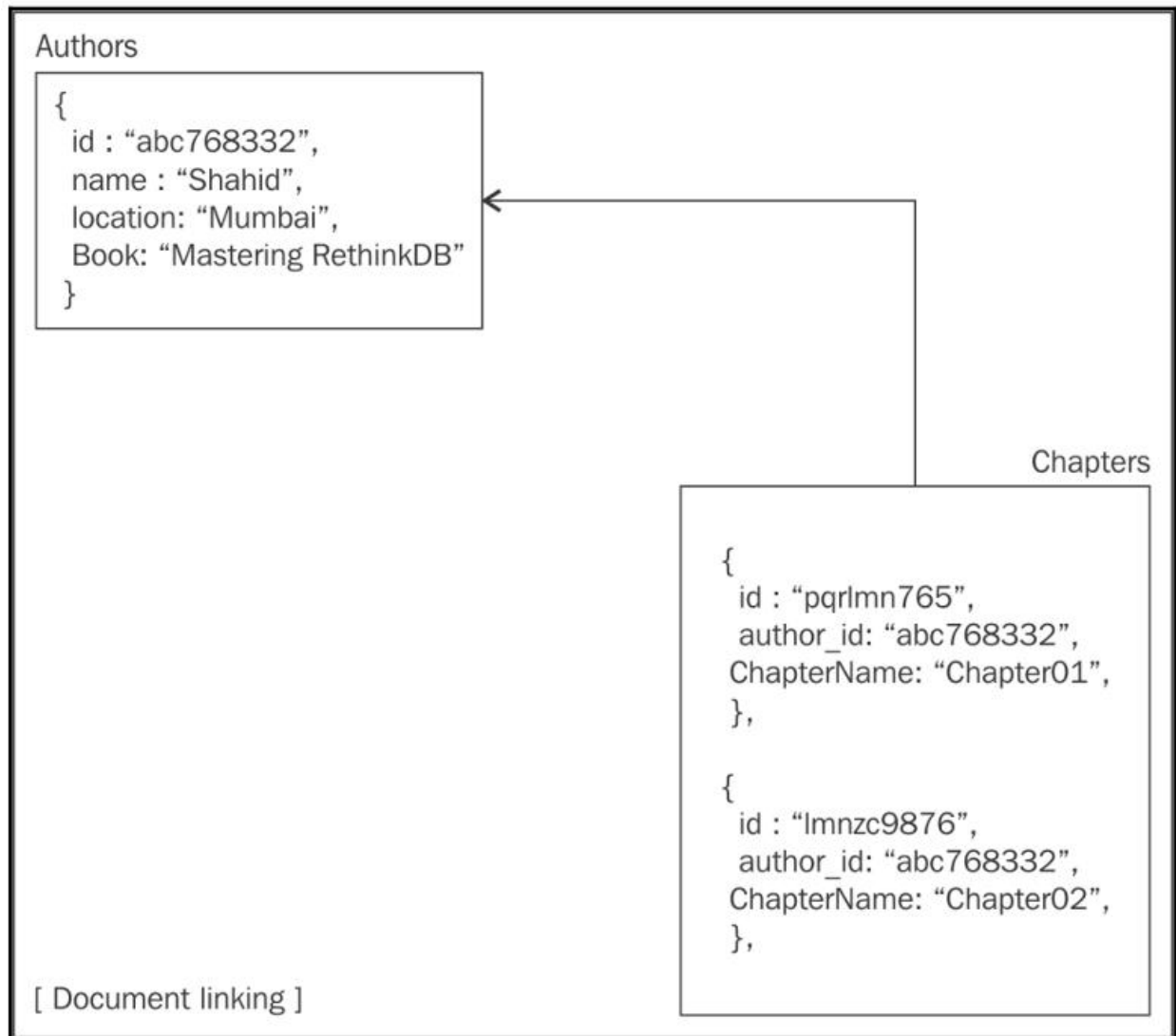
Hence, both pieces of data will be atomic.

Demerits of embedded arrays

As soon as the size of an embedded array exceeds a certain limit, *it costs too much computing power to load the entire array every time we have to perform an operation.*

Document linking in multiple tables

This is similar to the foreign key relationships we make in traditional SQL databases. Here, we link two or more tables using the index. We can also query the table using functions provided by RethinkDB to perform various joins (inner, outer, and so on)



Here is how we query the tables to perform JOINS:

```
r.db("books").table("chapters").filter({"author_id": "abc768332"}).run()
```

This will return every chapter for the author having ID as abc768332.

You can also use RethinkDB JOINS functions such as `eq_join()`;

Merits of document linking

The following merits are:

- *Data distribution* and very neat design
 - *No need to load all of the data* of chapters at once in order to perform any operation
 - *No matter what the size of documents* in chapters table is, *the performance won't be affected* as in embedded arrays
-

Demerits of document linking

The following demerits are:

- There are no foreign key constraints, so linking of data will be complicated.
 - In the case of an update in one table, it won't automatically update the data in another table.
-

Here is a list of JOIN commands provided by RethinkDB:

- `eq_join`
 - `innerjoin`
 - `outerjoin`
 - `Zip`
-